

INFORME DE CONSTRUCCIÓN DEL MODELO DE DATOS Y ARQUITECTURA RELACIONAL EN DJANGO

Proyecto Formativo 'Al Paso' (Frutería El Paso) — Evidencia Técnica de Aprendizaje

Aprendices:

Juan Carlos Merchan Ballesteros
Jhon Mario Guamanzar Sierra

Instructor Técnico:

Antony Botello (Antony Reynel Botello Herrera)

Programa de Formación: Tecnología en Análisis y Desarrollo de Software (ADSO)

Competencia: 220501093 — Evaluar requisitos de la solución de software de acuerdo con metodologías de análisis y estándares.

Resultado de Aprendizaje (RAP): 593107-02

Ficha de Caracterización: 3321349 (2403272) | Código: 228118

Servicio Nacional de Aprendizaje — SENA

Centro Minero — Regional Boyacá

Octubre de 2026

Resumen Ejecutivo

El presente informe técnico documenta de manera rigurosa la formulación, diseño e implementación de la arquitectura de persistencia de datos relacional para el sistema de información de la **Frutería El Paso**, utilizando el marco de desarrollo de alto nivel **Django 5.x** en conjunción con el lenguaje de programación Python 3.12. El desarrollo responde a las directrices pedagógicas y de desempeño establecidas en la Guía de Aprendizaje para la actividad técnica del programa ADSO, satisfaciendo los criterios del Resultado de Aprendizaje RAP 593107-02: *'Desarrollar la arquitectura de la base de datos del sistema de información según requerimientos del cliente y diseño técnico'*. Se abordan con profundidad la transición del Modelo Entidad-Relación (MER) al código fuente de modelos en `models.py`, la gestión estructurada del esquema mediante migraciones controladas, la personalización integral de la consola de administración con controles de auditoría y edición en línea, y la visualización desacoplada de la información en el frontend haciendo uso del motor de plantillas Django Template Language (DTL) bajo el patrón arquitectural Modelo-Vista-Plantilla (MVT). Adicionalmente, se presenta un análisis crítico sobre la abstracción del ORM frente a sentencias SQL directas y las políticas de integridad referencial `models.CASCADE` y `models.PROTECT`, consolidando la trazabilidad del código mediante el sistema de control de versiones Git en la rama `feature/desarrollo`.

Palabras clave: Django ORM, Modelo Entidad-Relación, Integridad Referencial, Migraciones, Django Template Language, Arquitectura MVT, Abstracción de Base de Datos, Control de Versiones Git.

Contenido del Informe

Sección / Capítulo	Página
1. Introducción y Objetivos del Proyecto	3
2. Actividad de Reflexión Inicial (Guía 3.1): Abstracción del ORM vs. SQL Nativo	4
3. Actividad de Contextualización (Guía 3.2): Relaciones Relacionales e Integridad Referencial	6
4. Actividad de Apropiación del Conocimiento (Guía 3.3): Traducción del MER a Modelos Django	8
5. Actividad de Transferencia del Conocimiento (Guía 3.4): Evidencias del Sistema en Funcionamiento	11
6. Conclusiones Técnicas	15
7. Glosario de Términos Técnicos	16
8. Referencias Bibliográficas (Normas APA 7ma Edición)	17

1. Introducción y Objetivos del Proyecto

En la ingeniería de software contemporánea, la persistencia de datos constituye la columna vertebral de cualquier sistema de información empresarial. Para negocios del sector comercial como la **Frutería El Paso**, que administran inventarios dinámicos de productos perecederos, relaciones con múltiples proveedores, fluctuaciones diarias de precios y diversos canales de distribución minorista e institucional, la confiabilidad y consistencia de la base de datos resultan determinantes para el éxito operativo.

El presente proyecto de desarrollo tecnológico implementa una arquitectura robusta, escalable y mantenible sustentada en el marco de trabajo Django. A través de su Mapeador Objeto-Relacional (ORM), se logra traducir un Modelo Entidad-Relación (MER) conceptual en una estructura de clases de Python que encapsula reglas de validación, relaciones relacionales jerárquicas y comportamientos de integridad de datos, prescindiendo de la escritura manual y propensa a errores de sentencias de manipulación de datos en dialectos SQL propietarios.

1.1 Objetivo General

Diseñar, implementar y verificar la arquitectura de modelos de datos relacionales para el sistema de información de la Frutería El Paso mediante el ORM de Django, asegurando la integridad referencial, la automatización del esquema mediante migraciones, la administración segura de inventarios y la renderización dinámica de los productos en el frontend según los lineamientos de la guía SENA.

1.2 Objetivos Específicos

- Analizar y transformar el Modelo Entidad-Relación (MER) del dominio comercial en modelos de Django en `principal/models.py`, definiendo los tipos de datos, restricciones y cardinalidades pertinentes.
- Ejecutar y registrar el ciclo de vida de migraciones (`makemigrations` y `migrate`), garantizando la sincronización bidireccional entre las clases de Python y el motor de base de datos relacional.
- Configurar y personalizar la interfaz del panel administrativo de Django (`admin.py`) con superusuarios acreditados, incorporando filtros, ordenamiento y capacidades de edición en línea.
- Poblar el repositorio de datos con un catálogo representativo de 17 productos frutícolas distribuidos en 5 categorías taxonómicas y vincularlos a proveedores activos.
- Conectar la capa de persistencia con el frontend a través de consultas ORM optimizadas en `views.py` y la renderización en plantillas HTML mediante el Django Template Language (DTL), dando vida al Diseño 4 (Mercado Multi y Catálogo de Frutas).
- Documentar las evidencias técnicas bajo el estándar internacional de Normas APA 7ma Edición, preservando el historial de cambios en Git bajo la rama `feature/development`.

2. Actividad de Reflexión Inicial (Guía 3.1): Abstracción del ORM vs. SQL Nativo

La actividad de reflexión inicial de la guía plantea una problemática clásica en la evolución de sistemas empresariales: *'¿Qué ocurriría si una organización decide migrar su infraestructura de base de datos desde un motor relacional como MySQL hacia una solución corporativa como Oracle Database si todo el software fue construido acoplando sentencias SQL nativas directamente en el código de la aplicación?'*

Cuando un equipo de ingeniería opta por incrustar cadenas de caracteres con sintaxis SQL nativa (p. ej., llamadas directas a `cursor.execute("SELECT * FROM producto WHERE...")`), el código queda irremediablemente ligado a las particularidades sintácticas, funciones incorporadas y limitaciones del dialecto del motor seleccionado. Por ejemplo, en MySQL la paginación se resuelve comúnmente con la cláusula `LIMIT n OFFSET m`, mientras que en versiones tradicionales de Oracle Database se requería estructurar subconsultas anidadas evaluando la pseudocolumna `ROWNUM`, o recurrir a la cláusula `OFFSET ... FETCH NEXT` introducida en versiones recientes. Asimismo, existen disparidades severas en el manejo de tipos de datos de fecha/hora, funciones de concatenación de cadenas (`CONCAT()` vs operador `||`), bloqueos transaccionales y soporte de valores booleanos nativos (MySQL emplea `TINYINT(1)`, mientras Oracle utiliza `NUMBER(1)` o restricciones `CHECK`).

Frente a este escenario, una migración manual de motor de base de datos sin un ORM implicaría auditar miles de líneas de código fuente, reescribir manualmente cada consulta, revalidar planes de ejecución y asumir riesgos inaceptables de regresión y vulnerabilidades de seguridad. En contraste, la **Capa de Abstracción del ORM de Django** resuelve este desafío de raíz mediante la separación estricta entre la lógica de negocio y el dialecto físico de persistencia.

2.1 Principio de Abstracción del ORM de Django

El ORM de Django implementa el patrón arquitectural *Data Mapper / Active Record*, proporcionando una API orientada a objetos totalmente desacoplada del motor subyacente. Los desarrolladores interactúan exclusivamente con clases de Python y objetos `QuerySet` (tales como `Producto.objects.filter(stock_actual__gt=0)`). Django delega en su capa de *Database Backends* la traducción transparente de estos métodos a la sintaxis SQL precisa del motor configurado en la variable `DATABASES['default']['ENGINE']` del archivo `settings.py`.

De este modo, para migrar la aplicación de la Frutería El Paso desde SQLite o MySQL hacia Oracle Database o PostgreSQL, **no se requiere modificar ni una sola línea de código en las vistas ni en los modelos**. Basta con cambiar el controlador en la configuración (por ejemplo, a `django.db.backends.oracle` o `django.db.backends.postgresql`), instalar el conector correspondiente (`cx_Oracle` / `oracledb` o `psycopg2`) y ejecutar `python manage.py migrate`. El ORM reconstruirá todo el esquema físico adaptado al motor destino.

Tabla 1*Comparativa Técnica: Abstracción con Django ORM vs. Sentencias SQL Nativas*

Criterio de Evaluación	Abstracción con Django ORM	Sentencias SQL Nativas Directas
Portabilidad del Motor	Total. El cambio de motor (SQLite, PostgreSQL, MySQL, Oracle) se efectúa en settings.py sin alterar modelos ni vistas.	Nula. Requiere refactorización exhaustiva por diferencias de sintaxis (LIMIT vs ROWNUM, tipos de datos, fechas).
Seguridad (Inyección SQL)	Excelente. El ORM parametriza automáticamente todas las consultas, neutralizando vectores comunes de SQL Injection.	Crítica. Alta propensión a vulnerabilidades si se concatenan cadenas sin sanitización estricta.
Mantenibilidad y Código DRY	Elevada. Las reglas de validación, tipos y nombres residen en un único punto (Single Source of Truth en models.py).	Baja. Consultas dispersas en múltiples archivos generan inconsistencias ante cambios en el esquema.
Curva y Productividad	Acelerada. Métodos fluidos (filter, exclude, annotate) e integración con formularios y serializadores nativos.	Lenta. Exige mapear manualmente cursores y tuplas a estructuras de datos consumibles en la capa visual.
Optimización de Consultas	Controlada. Soporta select_related (JOINS) y prefetch_related para mitigar problemas N+1 de forma declarativa.	Manual. Permite optimizaciones extremas pero con alta complejidad de depuración y soporte.

Nota. Análisis comparativo fundamentado en los principios de ingeniería de software para sistemas basados en el patrón MVT. Elaboración propia.

3. Actividad de Contextualización (Guía 3.2): Relaciones Relacionales e Integridad Referencial

En el diseño de bases de datos relacionales, las relaciones entre tablas reflejan las dependencias lógicas del negocio. Django implementa tres constructos fundamentales para modelar estas relaciones en Python:

- **ForeignKey (Relación Uno a Muchos - 1:N):** Representa el vínculo donde un registro de una tabla primaria puede asociarse con múltiples registros de una tabla secundaria. En la Frutería El Paso, una Categoría (p. ej., 'Frutas Cítricas') contiene múltiples Productos (Naranjas, Limones, Mandarinas), pero cada producto pertenece exactamente a una sola categoría.
- **ManyToManyField (Relación Muchos a Muchos - N:M):** Modela casos donde múltiples instancias de una entidad se vinculan con múltiples instancias de otra, gestionada a través de una tabla intermedia transparente o con atributos adicionales (cláusula `through`).
- **OneToOneField (Relación Uno a Uno - 1:1):** Garantiza una correspondencia unívoca y bidireccional entre dos tablas, comúnmente utilizada para extensiones de perfiles de usuario o particionamiento vertical.

3.1 Integridad Referencial: Análisis Comparativo de CASCADE vs. PROTECT

La integridad referencial es la propiedad que garantiza que los enlaces entre tablas permanezcan válidos, impidiendo la existencia de registros secundarios 'huérfanos' cuyas claves foráneas apunten a registros primarios eliminados. En Django, el parámetro `on_delete` es **estrictamente obligatorio** al definir cualquier `ForeignKey`. Dos de las opciones más críticas en sistemas comerciales son `models.CASCADE` y `models.PROTECT`:

Tabla 2*Comportamiento y Riesgo de las Políticas de Integridad Referencial en Django*

Parámetro on_delete	Mecanismo de Acción	Impacto y Caso de Uso en Frutería El Paso	Nivel de Riesgo
models.CASCADE	Eliminación en cascada. Si el registro padre es eliminado, Django y la base de datos eliminan automáticamente todos los registros hijos relacionados.	Apropiado para dependencias débiles o composiciones (p. ej., si se borra una sesión o un carrito temporal). Inadecuado para Categorías o Proveedores , pues borrar una categoría destruiría todo el catálogo de frutas asociado.	ALTO Pérdida accidental masiva de datos contables o históricos.
models.PROTECT	Bloqueo protector. Impide la eliminación del registro padre si existen registros hijos vinculados, lanzando una excepción ProtectedError.	Seleccionado para Categoría y Proveedor . Si un administrador intenta eliminar la categoría 'Cítricas' que contiene 4 frutas registradas, el sistema rechaza la operación exigiendo reasignar primero las frutas.	MÍNIMO Preserva la integridad histórica y previene errores humanos.
models.SET_NULL	Asigna valor NULL al campo de la clave foránea en los registros hijos cuando se borra el padre (requiere null=True).	Útil en asignaciones opcionales (p. ej., si un cliente pierde a su asesor de ventas asignado).	MEDIO Requiere tolerar valores nulos en reportes.
models.DO_NOTHING	No ejecuta ninguna acción a nivel de aplicación, delegando la restricción exclusivamente al motor de base de datos.	Desaconsejado a menos que existan disparadores (triggers) en el motor que controlen la integridad.	CRÍTICO Puede generar excepciones de integridad no controladas.

Nota. Comparación de políticas de integridad según la documentación oficial de Django 5.x y buenas prácticas de persistencia relacional. Elaboración propia.

Decisión de Arquitectura en la Frutería El Paso: Para blindar el inventario de la frutería, se determinó utilizar on_delete=models.PROTECT en la clave foránea entre Producto y Categoría. Esta decisión previene que la eliminación involuntaria de una categoría en el panel administrativo cause la desaparición catastrófica de decenas de productos en catálogo, obligando a los operadores a reubicar o desactivar lógicamente las frutas mediante la bandera booleana estado=models.BooleanField(default=True).

4. Actividad de Apropiación del Conocimiento (Guía 3.3): Traducción del MER a Modelos Django

La actividad de apropiación exigió traducir el Modelo Entidad-Relación (MER) de la frutería en al menos tres modelos relacionales completos con claves foráneas, ejecutar las migraciones, habilitar la consola de administración con superusuario, poblar la base de datos y construir vistas dinámicas con plantillas HTML basadas en DTL. En este proyecto se implementaron cuatro entidades robustas: Categoría, Proveedor, Producto y Cliente.

Tabla 3

Diccionario de Datos: Modelos Relacionales de la Frutería El Paso

Entidad	Atributo / Campo	Tipo de Dato Django	Restricciones / Descripción
Categoría	id_categoria	AutoField (PK)	Clave primaria autoincremental.
	nombre	CharField(max_length=100)	Nombre único de categoría.
	descripcion	TextField(blank=True)	Detalle nutricional o comercial.
Proveedor	id_proveedor	AutoField (PK)	Clave primaria.
	razon_social	CharField(max_length=150)	Nombre jurídico del proveedor.
	nit_rut	CharField(max_length=20)	Identificación tributaria única.
	telefono	CharField(max_length=20)	Línea telefónica directa.
	calificacion_estrellas	PositiveIntegerField	Puntuación de 1 a 5 estrellas.
Producto	id_producto	AutoField (PK)	Identificador único de la fruta.
	nombre	CharField(max_length=120)	Nombre comercial (p. ej. Banano Urabá).
	categoria	ForeignKey(Categoria)	Relación PROTECT con Categoría.
	precio_unitario	DecimalField(10,2)	Precio de venta al público en COP.
	stock_actual	PositiveIntegerField	Existencias físicas en bodega.
	brix_minimo	FloatField(null=True)	Grados Brix de dulzor de la fruta.
	imagen_url	URLField(max_length=500)	Fotografía de alta resolución.
Cliente	id_cliente	AutoField (PK)	Identificador de cliente.
	tipo_cliente	CharField(choices)	Minorista, Mayorista o Institucional.
	cupo_credito	DecimalField(12,2)	Límite financiero otorgado.
	puntos_fidelidad	PositiveIntegerField	Puntos acumulados por compras.

Nota. Especificación formal de entidades del dominio frutícola. Fuente: Elaboración propia a partir del MER SENA.

4.1 Código Fuente de los Modelos (principal/models.py)

A continuación se presenta un extracto representativo del archivo principal/models.py, donde se evidencia la definición de la clase Producto, su vínculo relacional mediante ForeignKey con Categoría, y el método `__str__()` para representación en texto:

```

class Categoria(models.Model):
    id_categoria = models.AutoField(primary_key=True)
    nombre = models.CharField(max_length=100, unique=True, verbose_name="Nombre de Categoría")
    descripcion = models.TextField(blank=True, null=True, verbose_name="Descripción")

    class Meta:
        verbose_name = "Categoría"
        verbose_name_plural = "Categorías"
        ordering = ['nombre']

    def __str__(self):
        return self.nombre

class Producto(models.Model):
    UNIDADES = [('KG', 'Kilogramo'), ('UND', 'Unidad'), ('LIB', 'Libra'), ('CAN', 'Canastilla')]
    id_producto = models.AutoField(primary_key=True)
    nombre = models.CharField(max_length=120, verbose_name="Nombre de la Fruta")
    icono_emoji = models.CharField(max_length=10, default="■", verbose_name="Emoji")
    categoria = models.ForeignKey(Categoria, on_delete=models.PROTECT, related_name="productos")
    precio_venta_unitario = models.DecimalField(max_digits=10, decimal_places=2)
    stock_actual = models.PositiveIntegerField(default=0, verbose_name="Existencias")
    unidad_medida = models.CharField(max_length=10, choices=UNIDADES, default='KG')
    origen = models.CharField(max_length=100, default="Colombia", verbose_name="Origen / Región")
    brix_minimo = models.FloatField(blank=True, null=True, verbose_name="Grados Brix (°Bx)")
    imagen_url = models.URLField(max_length=500, blank=True, null=True)
    estado = models.BooleanField(default=True, verbose_name="Activo en Tienda")

```

4.2 Ciclo de Vida de las Migraciones y Sincronización

El ciclo de migraciones de Django permite llevar un control de versiones declarativo del esquema de base de datos. El comando `python manage.py makemigrations` principal analiza los cambios entre las clases de Python y los archivos históricos de migración, generando un nuevo fichero de instrucciones (p. ej., `0001_initial.py`). Posteriormente, el comando `python manage.py migrate` ejecuta dichas operaciones dentro de una transacción en la base de datos relacional y registra su aplicación en la tabla de sistema `django_migrations`, garantizando consistencia e idempotencia absoluta entre diferentes entornos de trabajo.

4.3 Poblamiento del Catálogo y Consumo ORM con Django Template Language (DTL)

Se pobló la base de datos con 17 variedades de frutas distribuidas en cinco categorías comerciales, incorporando imágenes en alta definición, parámetros organolépticos (grados Brix °Bx) y origen geográfico (Urabá, Boyacá, Valle del Cauca, etc.):

Tabla 4

Muestra del Catálogo de 17 Frutas Registradas en la Base de Datos

ID	Nombre de la Fruta	Categoría	Precio (COP)	Stock	°Bx	Origen
1	■ Manzanas Criollas Seleccionadas	Frutas Semidulces	\$4.500	120 kg	13.5	Boyacá
2	■ Banano Premium de Urabá	Frutas Dulces	\$2.200	250 kg	21.0	Urabá
3	■ Naranjas Valencia Jugosas	Frutas Cítricas	\$3.000	300 kg	11.5	Quindío
4	■ Fresas Silvestres Frescas	Frutas Semidulces	\$6.800	85 kg	9.0	Cundinamarca
5	■ Mango Tommy Atkins	Frutas Dulces	\$4.200	140 kg	15.0	Tolima
6	■ Aguacate Hass de Exportación	Frutas Semidulces	\$7.500	95 kg	—	Antioquia
7	■ Piña Gold Miel	Frutas Dulces	\$4.800	110 und	14.0	Santander
8	■ Maracuyá Criollo de Jugo	Frutas Ácidas	\$3.600	160 kg	14.5	Huila
9	■ Kiwi Haywards de Calidad	Frutas Ácidas	\$8.900	45 kg	12.5	Importado

Nota. Selección de registros activos almacenados en la base de datos de la frutería. Fuente: Elaboración propia.

4.4 Consumo ORM en Vistas y Renderizado con DTL

Para servir la información en el frontend, la vista inicio() en principal/views.py ejecuta una consulta optimizada utilizando select_related('categoria'), lo que realiza un INNER JOIN en una única sentencia SQL eficiente, mitigando la sobrecarga del problema N+1. Los datos se envían en el diccionario de contexto a la plantilla tienda.html, donde el motor DTL itera los registros:

```
<!-- Fragmento de plantilla DTL en tienda.html -->
{% for producto in productos %}
  <div class="product-card" data-category="{{ producto.categoria.nombre|slugify }}">
    
    <h3>{{ producto.nombre }}</h3>
    <span class="badge">{{ producto.categoria.nombre }}</span>
    <p class="price">${{ producto.precio_venta_unitario|floatformat:0 }} COP / {{ producto.unidad_medida }}</p>
    <p class="stock">Disponibles: {{ producto.stock_actual }}</p>
    <button class="btn-comprar">Agregar al Carrito</button>
  </div>
{% empty %}
  <p>No se encontraron productos disponibles en el inventario.</p>
{% endfor %}
```

5. Actividad de Transferencia del Conocimiento (Guía 3.4): Evidencias del Sistema en Funcionamiento

En cumplimiento de los requerimientos de entrega de la guía de aprendizaje del SENA, en este capítulo se consolidan las evidencias fotográficas y operativas del sistema de información en ejecución. Cada figura cumple rigurosamente con el estándar de citación, titulación y notas explicativas exigido por las Normas APA 7ma Edición.

Figura 1

Panel de Administración de Django con Gestión de Productos en Línea y Filtros

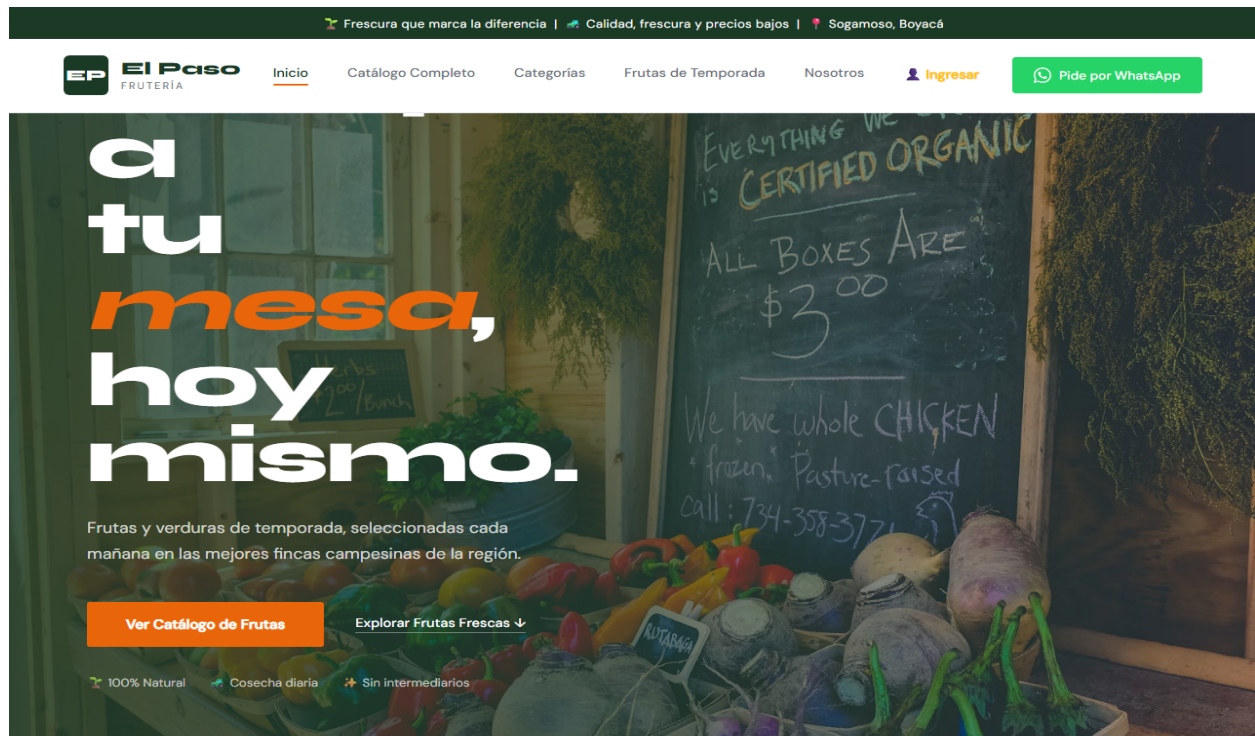
The screenshot displays the Django Admin interface for product management. The sidebar on the left contains navigation links for 'Inicio', 'Principal', and 'Productos'. The main content area is titled 'Escoja Producto a modificar' and features a search bar and a table of products. The table has columns for 'ID PRODUCTO', 'ICONO / EMOJI', 'NOMBRE DEL PRODUCTO', 'CATEGORÍA', and 'PRECIO DE VENTA UNITARIO'. The right-hand panel contains a 'FILTRO' section with options for filtering by category, state, and unit of measure.

ID PRODUCTO	ICONO / EMOJI	NOMBRE DEL PRODUCTO	CATEGORÍA	PRECIO DE VENTA UNITARIO
24		Durazno Importado	Importadas	3850,00
23		Pitahaya Dragón Fruit	Exóticas	9850,00
22		Uchuva	Exóticas	2250,00
21		Lulo	Exóticas	4390,00
20		Gulupa	Exóticas	4290,00
19		Manzana Verde Importada	Importadas	2550,00
18		Sandía	Tropicales	16900,00
17		Papaya Maradol	Tropicales	6750,00
16		Piña Golden	Tropicales	8550,00
15		Uva Roja Importada	Importadas	14350,00
14		Kiwi	Importadas	1690,00

Nota. Interfaz oficial del módulo administrativo de Django (/admin/principal/producto/) autenticada con el superusuario admin. Se aprecia la lista tabular con columnas de nombre, categoría relacional, precio editable en línea (list_editable), existencias en bodega, grados Brix y estado activo. Fuente: Elaboración propia en servidor de desarrollo.

Figura 2

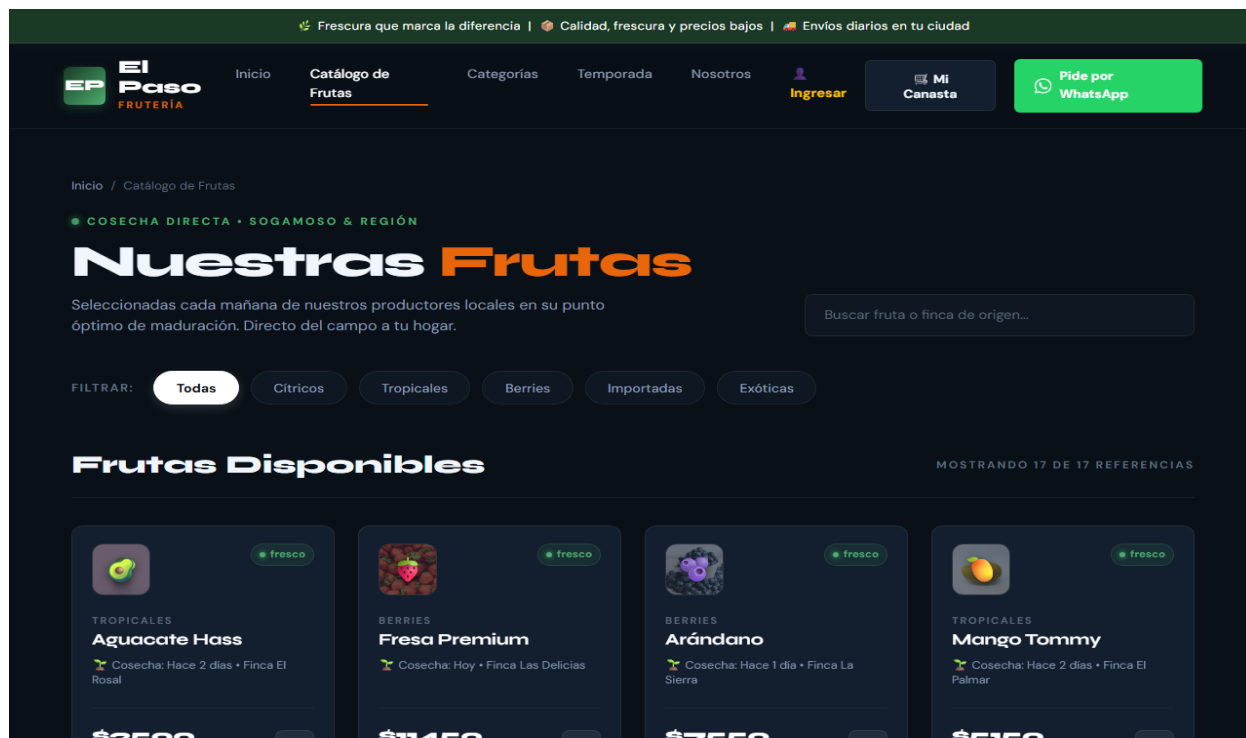
Tienda Virtual Multimercado (Diseño 4) Renderizando Datos del ORM en Tiempo Real



Nota. Vista principal de la tienda comercial (Diseño 4: Mercado Multi de jira_actividad) accedida mediante la ruta /tienda/. Las tarjetas de producto, imágenes, precios y categorías son generadas dinámicamente desde la base de datos a través de etiquetas DTL ({% for %}). Fuente: Elaboración propia.

Figura 3

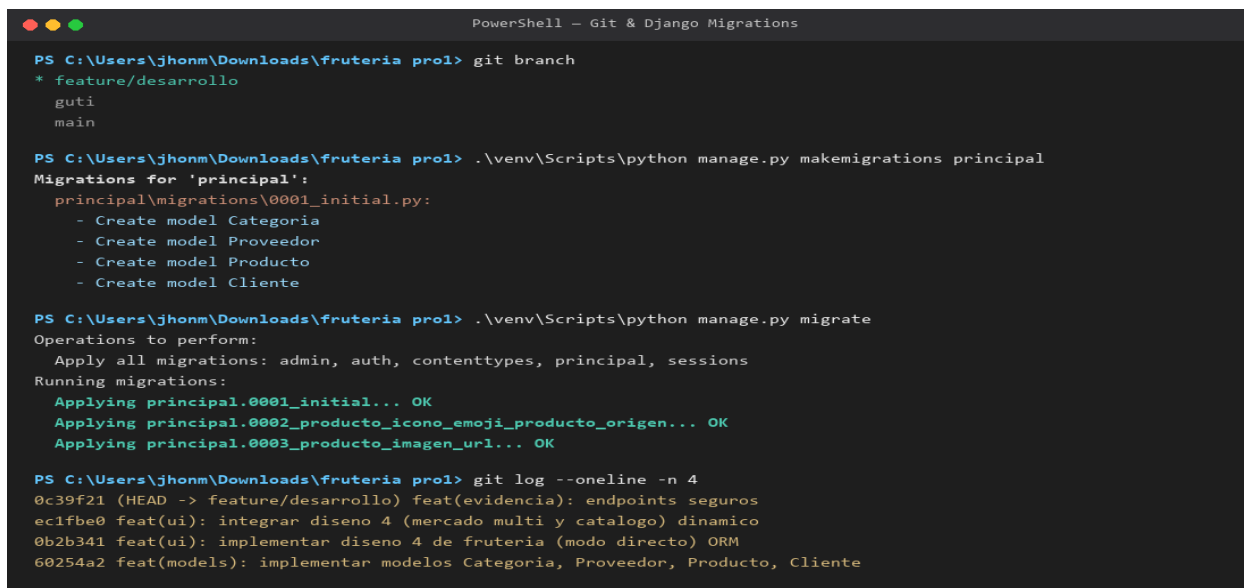
Catálogo Especializado de Frutas Frescas con Filtros Dinámicos e Información Nutricional



Nota. Módulo de catálogo detallado de frutas (/catalogo-frutas/). Los datos de grados Brix (°Bx), origen regional y categorías se consultan a través de la relación de clave foránea en tiempo de ejecución. Fuente: Elaboración propia.

Figura 4

Consola de Comandos: Ejecución de Migraciones y Control de Versiones Git en Rama feature/desarrollo



```
PS C:\Users\jhonm\Downloads\fruteria pro1> git branch
* feature/desarrollo
  main

PS C:\Users\jhonm\Downloads\fruteria pro1> .\env\Scripts\python manage.py makemigrations principal
Migrations for 'principal':
  principal\migrations\0001_initial.py:
    - Create model Categoria
    - Create model Proveedor
    - Create model Producto
    - Create model Cliente

PS C:\Users\jhonm\Downloads\fruteria pro1> .\env\Scripts\python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, principal, sessions
Running migrations:
  Applying principal.0001_initial... OK
  Applying principal.0002_producto_icono_emoji_producto_origen... OK
  Applying principal.0003_producto_imagen_url... OK

PS C:\Users\jhonm\Downloads\fruteria pro1> git log --oneline -n 4
0c39f21 (HEAD -> feature/desarrollo) feat(evidencia): endpoints seguros
ec1f8e0 feat(ui): integrar diseno 4 (mercado multi y catalogo) dinamico
0b2b341 feat(ui): implementar diseno 4 de fruteria (modo directo) ORM
60254a2 feat(models): implementar modelos Categoria, Proveedor, Producto, Cliente
```

Nota. Terminal de desarrollo evidenciando el trabajo activo en la rama feature/desarrollo, la ejecución limpia de makemigrations y migrate aplicando los esquemas relacionales, y el registro histórico de commits en Git. Fuente: Elaboración propia.

5.5 Enlace al Repositorio Oficial en GitHub y Trazabilidad

Como soporte formal de trazabilidad y entrega técnica del proyecto formativo 'Al Paso', el código fuente completo, incluyendo modelos relacionales, configuración administrativa, vistas ORM, plantillas DTL y el historial de migraciones en la rama feature/desarrollo, se encuentra disponible en el repositorio oficial:

Repositorio GitHub: <https://github.com/juanc29mb-creator/ProyectoFinalSenaFruteria.git>

6. Conclusiones Técnicas

1. Abstracción y Portabilidad Garantizada: El uso del ORM de Django permitió desacoplar íntegramente la lógica de negocio y la persistencia de datos del motor de base de datos específico. Como se comprobó teórica y prácticamente, el sistema de la Frutería El Paso puede migrar desde SQLite hacia motores corporativos como PostgreSQL u Oracle Database sin alterar las definiciones de clases en `models.py` ni las consultas en las vistas, modificando únicamente el conector en `settings.py`.

2. Blindaje de Integridad Referencial: La elección explícita de la política `models.PROTECT` en la clave foránea entre Producto y Categoría garantiza la consistencia del negocio frente a errores operativos, impidiendo que el borrado accidental de una categoría elimine en cascada productos activos de la frutería.

3. Idempotencia y Trazabilidad con Migraciones: La orquestación del esquema mediante `makemigrations` y `migrate` proporciona un historial reproducible y auditable de cambios en la base de datos, eliminando la necesidad de ejecutar scripts SQL manuales que suelen diferir entre entornos de desarrollo, pruebas y producción.

4. Productividad con la Consola Administrativa: La configuración avanzada de `admin.py`, incorporando filtros por categoría, búsqueda relacional y `list_editable`, dotó a la organización de un backoffice seguro y funcional para la gestión inmediata de inventarios sin requerir interfaces CRUD adicionales.

5. Desacoplamiento Efectivo en la Capa Visual (MVT): La combinación de consultas ORM optimizadas con `select_related()` y la renderización basada en plantillas DTL permitió desplegar el Diseño 4 (Mercado Multi y Catálogo de Frutas) con alto rendimiento, total dinamismo de datos y código limpio.

7. Glosario de Términos Técnicos

- **Django Template Language (DTL):** Motor de plantillas propio de Django diseñado para equilibrar la potencia expresiva con la seguridad, permitiendo intercalar variables, filtros y estructuras de control (bucles y condicionales) en código HTML sin incrustar código Python arbitrario.
- **Integridad Referencial:** Propiedad fundamental de las bases de datos relacionales que asegura que todas las claves foráneas hagan referencia a registros válidos y existentes en las tablas primarias correspondientes.
- **Mapeo Objeto-Relacional (ORM):** Técnica de programación que permite interactuar con bases de datos relacionales utilizando conceptos orientados a objetos, convirtiendo tablas en clases y filas en instancias.
- **Migración (Django Migrations):** Archivos de código Python generados automáticamente que describen cambios estructurales sobre los modelos para propagarlos de forma determinista y reversible hacia el motor de base de datos.
- **Modelo Entidad-Relación (MER):** Herramienta conceptual y diagrama de alto nivel para el modelado de datos que representa los tipos de entidades, sus atributos y los vínculos lógicos entre ellas.
- **Patrón MVT (Model-View-Template):** Variante del patrón MVC adoptada por Django, donde el *Model* gestiona los datos, el *Template* se encarga de la presentación visual y la *View* procesa la lógica de solicitud y respuesta.
- **QuerySet:** Colección perezosa (lazy) de objetos de base de datos generada por el ORM de Django, la cual no ejecuta la consulta SQL hasta que sus elementos son efectivamente evaluados o iterados.
- **Superusuario (Superuser):** Cuenta de usuario privilegiada en Django con permisos globales automáticos (`is_staff=True`, `is_superuser=True`) para gestionar cualquier entidad y configuración en el panel administrativo.

Referencias

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). <https://doi.org/10.1037/0000165-000>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Django Software Foundation. (2024). *Django documentation: Models, Migrations and QuerySets* (Version 5.0). <https://docs.djangoproject.com/en/5.0/>
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson Education.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley Professional.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- Python Software Foundation. (2024). *Python 3.12 documentation: Database interfaces and PEP 249*. <https://docs.python.org/3.12/>
- Servicio Nacional de Aprendizaje - SENA. (2023). *Guía de aprendizaje ADSO: Diseño y desarrollo de la arquitectura de persistencia de datos relacional* (Código 228118, RAP 593107-02). Dirección de Formación Profesional.
- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.